



MULTI-SERVIDOR

CHADWICK CARRETO ARELLANO

28 DE SEPTIEMBRE DE 2026

ROSALES BARRAZA ERICK EFREN
7CM4

El almacén como sistema distribuido

Cuarta parte: inventario particionado entre varios servidores, con enrutamiento y consultas agregadas

Antecedentes

Las tres partes anteriores del proyecto giraron alrededor de un solo servidor que guardaba todo el inventario de la tienda. Primero se modeló como un problema de planificación de procesos, con Round Robin y semáforos. Después ese mismo modelo se expuso por dos vías de transporte distintas, un servidor web con WebSocket y un servidor de sockets TCP. Más adelante se identificó a cada cliente conectado y sus acciones quedaron etiquetadas en una bitácora compartida.

En todas esas versiones, sin importar cuántos clientes se conectaran, el almacén vivía en un único lugar: un solo proceso con todo el stock. Lo que se pidió ahora es distinto: que el almacén mismo deje de ser una sola pieza y se convierta en el sistema distribuido, repartido entre varios servidores a los que cualquier cliente pueda conectarse.

Planteamiento del problema

El reto tenía dos partes. La primera, que hubiera de verdad más de un servidor, cada uno responsable de una porción distinta del inventario de la tienda, y no solo varias copias del mismo proceso corriendo en paralelo. La segunda, que un cliente conectado a cualquiera de esos servidores pudiera pedir o consultar cualquier producto del catálogo completo, incluso uno que vive en otro servidor, sin tener que saber de antemano quién lo tiene.

Ese segundo punto es el problema real de un sistema distribuido: en cuanto el inventario queda repartido entre máquinas distintas, una petición que llega al servidor equivocado tiene que encontrar su camino hacia el correcto, y una consulta del inventario completo tiene que salir a preguntarle a cada servidor por separado y juntar las respuestas antes de contestar.

Solución

El inventario se dividió entre tres nodos, cada uno un proceso de Node.js independiente que escucha en su propio puerto TCP y es dueño exclusivo del stock de un producto.

Nodo	Nombre	Puerto	Producto propio
N1	Almacén Norte	6001	Manzanas
N2	Almacén Centro	6002	Peras
N3	Almacén Sur	6003	Bananas

Cada nodo conoce, desde un archivo de configuración compartido, qué producto le pertenece a cada uno de los demás. Con eso le basta para decidir, sin preguntarle a nadie, si un pedido que le llega es suyo o de otro nodo.

Cuando un pedido llega a un nodo que no es dueño de ese producto, ese nodo abre su propia conexión TCP hacia el nodo dueño, le reenvía el pedido, espera la respuesta y se la entrega al cliente. El cliente nunca se entera de que su propio nodo tuvo que consultar a otro: desde su punto de vista, pidió un producto y le contestaron.

Un cliente también puede pedirle a cualquier nodo el inventario completo de la tienda. Ese nodo contesta con su propio stock y, al mismo tiempo, abre una conexión con cada uno de sus compañeros para pedirles el suyo, junta todas las respuestas y entonces responde. Este patrón, mandar una pregunta que se reparte entre varios nodos y juntar las respuestas al final, es habitual en las consultas de un sistema distribuido.

Como cada producto vive en un solo nodo, ese nodo es el único que toca su propio stock. Dos nodos nunca compiten por la misma variable, así que no hizo falta un bloqueo distribuido ni un protocolo de acuerdo entre máquinas: repartir los datos de esta forma ya evita esa carrera por sí solo. Dentro de un mismo nodo, la operación local sigue envuelta en el mismo semáforo usado en las partes anteriores, aunque vale aclarar que un proceso de Node.js ya es de un solo hilo, así que un bloque de código síncrono como este no puede ser interrumpido por otra tarea mientras corre. El semáforo se dejó de todas formas, tanto por mantener el mismo patrón de las partes anteriores como para proteger ese bloque si más adelante se vuelve asíncrono.

Implementación

- distribuido/config.js guarda la lista de nodos: su identificador, nombre, dirección, puerto y el producto que le pertenece a cada uno. Es el mismo archivo que importan todos los nodos y el cliente, así que todos comparten la misma idea de quién es dueño de qué.
- distribuido/warehouse-node.js es el servidor de cada nodo. Reutiliza sin cambios Semaphore e Inventory de las partes anteriores para proteger su propio stock, y agrega la lógica de enrutamiento hacia los demás nodos y de reunir sus respuestas en una consulta de inventario completo.
- distribuido/warehouse-client.js es un cliente de consola que se conecta a un nodo por su identificador (N1, N2 o N3) y manda los comandos order producto cantidad y stock, mostrando siempre si el pedido se atendió en el mismo nodo o se enrutó hacia otro.

La parte que decide si un pedido es local o hay que reenviarlo queda así:

warehouse-node.js — enrutamiento de un pedido según su dueño

```
if (msg.cmd === 'order') {
  const owner = ownerOf(msg.product);
  if (!owner) {
    send(socket, { type: 'error', message: `Producto desconocido: ${msg.product}` });
    return;
  }

  let result;
  if (owner.id === self.id) {
    result = processLocalOrder(msg.product, msg.qty);
  } else {
    result = await rpcCall(owner, { cmd: 'peerOrder', product: msg.product, qty: msg.qty,
fromNode: self.id });
  }

  const payload = { type: 'orderResult', handledBy: owner.id, routedVia: self.id, product:
msg.product, qty: msg.qty, ...result };
  broadcastLocal(payload);
  return;
}
```

La consulta de inventario completo reparte la pregunta entre todos los nodos y junta las respuestas antes de contestar:

warehouse-node.js — consulta agregada de inventario entre nodos

```
if (msg.cmd === 'stock') {
  const byNode = { [self.id]: { nodeId: self.id, nodeName: self.name, stock:
inventory.snapshot() } };
  const results = await Promise.all(peers.map(p => rpcCall(p, { cmd: 'peerStock' })));
  for (const r of results) byNode[r.nodeId] = r;
  send(socket, { type: 'stock', byNode });
  return;
}
```

Ambos casos usan la misma función de bajo nivel para hablar con un nodo vecino: abre una conexión nueva, manda una línea de JSON, espera una línea de vuelta y cierra.

warehouse-node.js — llamada a un nodo vecino

```
function rpcCall(peer, payload) {
  return new Promise((resolve) => {
    const conn = net.connect(peer.port, peer.host, () => {
      conn.write(JSON.stringify(payload) + '\n');
    });
    conn.setEncoding('utf8');
    let buffer = '';
    conn.on('data', (chunk) => {
      buffer += chunk;
      const idx = buffer.indexOf('\n');
      if (idx >= 0) {
        resolve(JSON.parse(buffer.slice(0, idx)));
        conn.end();
      }
    });
    conn.on('error', () => resolve({ ok: false, error: `nodo ${peer.id} no disponible`
}));
  });
}
```

Resultados

- Un cliente conectado al nodo N1 pidió manzanas, que son de N1, y el pedido se despachó de forma local, sin pasar por ningún otro nodo.
- El mismo cliente pidió peras, que pertenecen a N2, y el pedido se despachó de todas formas: la respuesta llegó marcada como enrutado por N1 hacia N2, dejando claro que hubo un salto de red de por medio aunque el cliente nunca se conectó directamente a N2.
- Un segundo cliente, conectado al mismo tiempo al nodo N3, pidió bananas de forma local y después consultó el inventario completo: vio reflejado, en tiempo real, el descuento de manzanas y peras que había hecho el primer cliente en los otros dos nodos, sin que su propio nodo tuviera esa información de antemano.

- Un pedido de 999 peras desde el nodo N3 se enrutó hacia N2 y volvió correctamente rechazado por falta de stock, probando que el mecanismo de enrutamiento también transporta bien los rechazos, no solo los pedidos exitosos.
- Durante la primera prueba apareció un error real: cada resultado de pedido llegaba dos veces al cliente, porque el nodo se lo mandaba directo y además lo repetía al difundirlo entre sus clientes locales. Se corrigió dejando solo la difusión, ya que el cliente que hizo el pedido siempre queda incluido en ella.

Conclusiones

El cambio central de esta parte fue dejar de pensar en un almacén con varios clientes para pensar en un almacén repartido entre varios servidores, cada uno responsable de una porción del catálogo. Separar quién es dueño de un producto de a qué nodo se conectó el cliente es, en el fondo, el problema que resuelve cualquier sistema con datos particionados.

Haber mantenido el semáforo y el inventario de las partes anteriores como piezas independientes de cualquier capa de transporte volvió a rendir frutos: se reutilizaron sin cambiar una sola línea dentro de cada nodo nuevo.

Repartir el inventario por producto evitó tener que construir un bloqueo distribuido, porque ningún dato tiene más de un dueño posible. La contraparte es una debilidad real que vale la pena dejar anotada: si un nodo se cae, su porción del catálogo deja de estar disponible para todo el sistema, sin que los otros nodos puedan cubrirlo. Resolver eso, con réplicas del mismo producto en más de un nodo o con un mecanismo de respaldo, sería el siguiente problema natural si el proyecto sigue creciendo.